

SPRINT 6 BACKLOG

Investment Manager - Cotações Externas e Cache

Sprint Goal

Integrar a aplicação a uma fonte externa de cotações de mercado, utilizar cache para reduzir chamadas à API e enriquecer as posições da carteira com valor atual, lucro/prejuízo e rentabilidade.

1. Natureza deste documento

Este Sprint Backlog foi reconstruído retrospectivamente a partir do histórico oficial e dos registros técnicos da Sprint 6. User Stories, critérios de aceite e Definition of Done foram formalizados a partir das entregas efetivamente registradas, sem atribuir à Sprint funcionalidades posteriores.

2. Objetivo da Sprint

Integrar a aplicação a uma fonte externa de cotações de mercado, utilizar cache para reduzir chamadas à API e enriquecer as posições da carteira com valor atual, lucro/prejuízo e rentabilidade.

3. Escopo funcional e técnico

- Criar feature quote e integrar com a API brapi por Bearer Token configurado em BRAPI_TOKEN.
- Criar BrapiClient, DTOs externos, QuoteService, QuoteController, QuoteResponse e QuoteNotFoundException.
- Adicionar Redis 7 via Docker Compose e cache de cotações por ticker.
- Usar chave quote:{ticker}, TTL de 5 minutos e fluxos cache HIT/MISS.
- Criar PositionMarketResponse e valorizar posições com cotação atual.
- Calcular currentValue, lucro/prejuízo não realizado e rentabilidade percentual.
- Manter cálculo contábil de posição separado da integração externa.
- Resolver lazy loading pontualmente com @EntityGraph, preservando associações LAZY.

4. Endpoints / superfícies principais

Tipo	Endpoint / rota	Finalidade
GET	/api/v1/quotes/{symbol}	Consultar cotação
GET	/api/v1/portfolios/{portfolioId}/positions	Consultar posição contábil
GET	/api/v1/portfolios/{portfolioId}/positions/market	Consultar posição valorizada a mercado

5. User Stories reconstruídas

US-06.1 - Consultar cotação

Como investidor, quero consultar o preço atual de um ativo.

US-06.2 - Valorizar carteira

Como investidor, quero ver valor atual, resultado e rentabilidade das posições.

US-06.3 - Reduzir chamadas externas

Como sistema, quero reutilizar cotações recentes em cache para reduzir dependência da API externa.

6. Critérios de aceite reconstruídos

- Cotação válida é obtida da brapi quando não existe cache.
- Cache HIT evita chamada externa e cache MISS salva a cotação com TTL.
- Ticker inexistente ou resposta vazia é tratado adequadamente.
- Posições zeradas não são consultadas no serviço de cotação.
- Rentabilidade evita divisão por zero e usa precisão adequada.
- Regras contábeis internas não dependem de Redis/internet.
- Suíte automatizada permanece verde.

7. Conceitos e competências trabalhados

- RestClient, Bearer Token e variáveis de ambiente.
- Redis, TTL, cache HIT/MISS e StringRedisTemplate.
- BigDecimal e precisão financeira.
- Lazy loading, Hibernate Proxy, LazyInitializationException e @EntityGraph.
- Separação entre domínio e integração externa.

8. Testes e validação

Resultado final registrado: 43 testes, 0 falhas, 0 erros, BUILD SUCCESS.

9. Fora do escopo

- Histórico de cotações.
- Usuários/JWT.
- Frontend.
- Evolução patrimonial histórica.

10. Definition of Done reconstruída

- Escopo principal da Sprint implementado e funcional.
- Regras de negócio da Sprint validadas.
- Testes previstos executados sem regressões relevantes.
- Integrações e endpoints/superfícies da Sprint validados.
- Código versionado conforme o fluxo Git do projeto.

11. Status final

Status: CONCLUÍDA. A Sprint 6 entregou o objetivo registrado para *Cotações Externas e Cache*.